

SIMD-X: Executor-Inclusion Proofs (EIP) for Validator-Side Deterministic Settlement Using Pedersen-Vector Commitments and POPK

Blessed Tosin-Oyinbo (tnxl)

April 2025

Abstract

This document presents a foundational commitment and proof design intended for research and discussion within the Solana protocol ecosystem. It proposes a cryptographic mechanism, based on Pedersen-Vector Commitments (PVCs) enhanced with Proofs of Partial Knowledge (POPK), that enables validators verify selective execution outcomes produced by external or modularized runtimes without then need to replay transaction states or require full transaction re-execution.

The core contribution of this mechanism is a construction that supports:

- Fixed-size inclusion attestations for arbitrary subsets of committed vectors
- Stateless and parallelizable validator-side verification routines
- Cryptographic bindings to PoH slot identifiers and execution results

The proposed mechanism is intended as a research draft and is not yet production-ready. It has not undergone any formal crypto analysis, performance benchmarking, or security audit. Its goal is solely to initiate discussion around lightweight proof-carrying execution for Solana and related modular settlement systems.

Disclaimer

This document is a research draft and should not be interpreted as a final specification or implementation plan. The constructions described in this document are mathematically valid but practically experimental in their design scope and integration assumptions. No formal security proofs are presented, and implementation in production systems should only proceed with appropriate cryptographic review.

Motivation

Solana’s architecture delivers high-throughput execution through deterministic runtime assumptions and full validator-side replay. However, as modular runtimes and delegated execution pipelines evolve, new questions arise around validator trust, inclusion guarantees, and proof-carrying verification.

This proposal aims to address a foundational gap in the protocol’s verifiability surface. Specifically: how can a validator verify that a set of transaction results was derived from a valid execution path, without re-running computation, without trusting external runtime logic, and without introducing sequential verification bottlenecks?

Traditional Merkle-based approaches, while being actually sound, require $O(\log n)$ inclusion proofs, sequential hash checks, and are not naturally parallelizable in validator environments.

We propose a succinct alternative using Pedersen-Vector Commitments (PVCs) and a Fiat-Shamir-based refinement called Proofs of Partial Knowledge (POPK). The scheme would enable validators verify inclusion of any subset of state transitions within a committed vector, using purely algebraic, stateless logic suitable for batch execution or SIMD-style parallelism.

Rather than being a drop-in module, this proposal outlines a primitive commitment language for inclusion attestations, meant to inform future SIMDs, protocol upgrades, or zk-augmented execution bridges.

Technical Construction

We define a vector commitment and selective proof protocol over a prime-order cyclic group $\mathbb{G}$ with generators $\{g_1, \dots, g_n\}, h$ derived via a deterministic, trapdoor-free method (e.g., hash-to-curve). These generators define a commitment language $\mathcal{L}_C$ over vectors of execution outputs.

Execution Outputs as Vectors

Let $\vec{x} = (x_1, \dots, x_n) \in \mathbb{Z}_q^n$ be the vector of execution results for a transaction batch computed by a modular runtime. Each x_i may represent a state delta, log hash, instruction result, or commitment to account-level changes.

We associate $\vec{x}$ with a PoH slot identifier s (i.e., the execution context), and define the commitment:

$$C := \sum_{i=1}^n x_i \cdot g_i + r \cdot h$$

where $r \in \mathbb{Z}_q$ is a blinding scalar chosen uniformly at random by the executor.

The pair (C, s) serves as the canonical execution commitment: it binds a specific state result vector to the temporal slot where execution occurred.

Partial Opening via Proofs of Partial Knowledge (POPK)

Given a subset of indices $J \subseteq [n]$, the goal is to prove that the values $\{x_j\}_{j \in J}$ are indeed part of the committed vector $\vec{x}$ without revealing the remaining entries.

Prover:

1. Compute:

$$C' := C - \sum_{j \in J} x_j \cdot g_j$$

This removes known openings, yielding a residual commitment to $\vec{x}_{\bar{J}}$ (the hidden portion).

2. Sample random $t \in \mathbb{Z}_q$ and compute ephemeral commitment:

$$T := t \cdot h$$

3. Compute Fiat–Shamir challenge:

$$c := \text{Hash}(C', T, s)$$

4. Respond with:

$$s := t + c \cdot r \pmod{q}$$

The proof is $\pi := (J, \{x_j\}_{j \in J}, T, s)$.

Verifier:

1. Reconstruct residual commitment:

$$C' := C - \sum_{j \in J} x_j \cdot g_j$$

2. Recompute challenge:

$$c := \text{Hash}(C', T, s)$$

3. Verify:

$$s \cdot h \stackrel{?}{=} T + c \cdot C'$$

Correctness Sketch

Assuming honest construction:

$$s \cdot h = (t + c \cdot r) \cdot h = t \cdot h + c \cdot r \cdot h = T + c \cdot (C - \sum_{j \in J} x_j \cdot g_j) = T + c \cdot C'$$

Hence, the verifier equation holds under binding correctness and group assumptions.

Commitment Language $\mathcal{L}_C$

We define:

$$\mathcal{L}_C = \left\{ C \in \mathbb{G} \mid \exists \vec{x}, r : C = \sum x_i \cdot g_i + r \cdot h \right\}$$

The goal of POPK is to prove that a subset of $\vec{x}$ belongs to a witness for $C \in \mathcal{L}_C$.

Proof Size and Parallelism

- Proof size: One group element T and one scalar s , regardless of $|J|$
- Verification is stateless and can be executed in SIMD lanes with precomputed scalar multipliers
- Suitable for bank-stage proof validation or leader verification logic

Security and Binding Guarantees

Hiding

The inclusion of the random scalar r and independent generator h ensures sufficiently adequate hiding of all unopened values in $\vec{x}$. Without knowledge of r , no information can be extracted about $\vec{x}_J$ from C or C' .

Binding

Pedersen commitments are binding under the discrete logarithm assumption in the group $\mathbb{G}$. Given that all generators $\{g_1, \dots, g_n, h\}$ are independently generated and fixed, it is computationally infeasible to find two different openings $(\vec{x}, r)$ and $(\vec{x}', r')$ such that:

$$\sum x_i \cdot g_i + r \cdot h = \sum x'_i \cdot g_i + r' \cdot h$$

This ensures vector commitments cannot be altered "changed" post-generation.

Soundness and Non-Malleability

POPK relies on the soundness of the Sigma protocol over the residual commitment C' , strengthened via the Fiat–Shamir heuristic. Since the challenge c is derived from (C', T, s) , the transcript is bound to the commitment and slot, rendering replay or re-binding attacks ineffective.

The use of s ensures that a forged proof must implicitly solve the discrete log relation:

$$T + c \cdot C' = s \cdot h$$

which is infeasible without knowledge of r .

Threat Mitigation Notes

- **Generator Trapdoors:** All generators $\{g_i\}, h$ must be derived via verifiable hash-to-curve methods to ensure that generator generation is trapdoor-free.
- **Blinding Reuse:** A fresh r must be generated per commitment. Reuse of r across distinct commitments could cause correlation or leakage attacks.
- **Replay Protection:** Each proof must include a temporal binding, e.g., a PoH slot s , in its Fiat–Shamir input to ensure uniqueness and prevent replay across slots or commitments.

Integration Considerations

This design is intended as a cryptographic primitive, not an end-to-end runtime framework. However, there are multiple plausible integration paths within Solana’s architecture:

1. Execution Output Commitment

Modular runtimes (such as those outside the Sealevel VM) can commit to transaction outputs as a vector $\vec{x}$ per slot. This commitment C is submitted along with metadata (e.g., instruction hashes, state deltas) to the bank or PoH recorder.

2. Proof Carrying Attestation

When a validator or execution node proposes a block, it includes:

- The commitment C
- Slot s binding the execution context
- Selective POPK proofs π for transactions of interest (e.g., external queries or check-pointed state)

3. Validator-side Verification

At replay or vote stages, validators verify π using fixed-base scalar multiplication over cached generators. Since the logic is stateless and linear, it is potentially parallelizable across GPU lanes.

4. zk-SNARK Compression (Optional)

For high-density proofs, the full POPK transcript may be wrapped in a zero-knowledge SNARK that attests to correct openings and adherence to application constraints (e.g., Merkle path, WASM execution).

5. Audit Trail and Light Client Use

POPK commitments can work as long-term proofs of state transition history, optionally coupled with Merkle roots for off-chain log replay or inclusion by light clients.

Conclusion and Future Directions

This document presents a foundational cryptographic scheme for vector commitments and selective proofs in the context of Solana’s high-throughput, slot-based architecture. By combining Pedersen-Vector Commitments with Proofs of Partial Knowledge (POPK), we enable validators to verify transaction result inclusion without re-execution or state replay.

Key benefits include:

- Stateless verification using only scalar operations
- Algebraic soundness and discrete-log security assumptions
- Compatibility with PoH slot identifiers and modular execution pipelines

This approach provides a minimal, auditably efficient way to introduce cryptographic attestation into runtime output handling. It does not aim to prescribe execution format, runtime logic, or state model, instead it offers a building block that may underpin future SIMD proposals or execution-verification bridges within Solana or derivative architectures.

Future Work

Areas for expansion include:

- Formal modeling of the proof system as a non-interactive argument
- Security proofs under standard assumptions (knowledge soundness, binding under adaptive adversaries)
- Performance benchmarking for parallelizable verification under GPU or validator-level constraints
- Compression of proof transcripts using inner-product arguments or zero-knowledge SNARKs
- Design patterns for executor-registration, public generator derivation, and proof indexing

Discussion Prompt

How should Solana evolve toward native support for proof-carrying execution in modular runtimes? Would an execution output commitment standard improve verifiability, inclusion guarantees, or validator-side cost models? What constraints or incentives are required to ensure honest inclusion without sacrificing throughput?